SYNECHRON L2 — CI/CD + DEVSECOPS 6-STAGE CHEAT SHEET

Interview-ready: Local Validation → CI → Artifact → Non-Prod → Production → Continuous Feedback

1. CODE VALIDATION & PRE-COMMIT — Developer Machine

Objective: Catch issues before code reaches the central repository.

Say this: I start with local validation. Before committing, developers run linting, secret detection and IaC security checks. Examples are ESLint/Black, gitleaks or trufflehog, and tfsec/Checkov. A clean local commit is the gate to the PR.

Remember: linting • pre-commit • secrets • IaC scan • shift-left

2. CI & PULL REQUEST — Automated Quality Gate

Objective: Prove the proposed change is safe and does not break existing functionality.

Say this: When a PR opens, CI runs dependency/SCA scanning, SAST, unit and integration tests, coverage checks and Terraform plan. Peer review plus successful automated checks are the merge gate.

Remember: SCA • SAST • tests • coverage • terraform plan • peer approval

3. ARTIFACT BUILD & DELIVERY — Immutable Package

Objective: Produce one verified, version-controlled artifact for promotion.

Say this: After merge, I build the production artifact, create the container image, scan it for vulnerabilities and publish it to a registry such as ECR. I identify it with the Git commit SHA and preferably deploy by immutable digest. The principle is build once and promote the same artifact.

Remember: immutable artifact • Docker • image scan • ECR • SHA • digest

4. CONTINUOUS DEPLOYMENT — NON-PRODUCTION

Objective: Validate the artifact in a realistic environment before production.

Say this: The pipeline deploys the same artifact to staging, test or UAT. Infrastructure is reconciled as code, DAST checks the running endpoint, and functional, end-to-end and performance tests validate the system. Compliance approval can be a production gate.

Remember: staging • IaC • DAST • E2E • performance • UAT

5. PRODUCTION DEPLOYMENT & VERIFICATION

Objective: Release safely while protecting live customer transactions.

Say this: For production I prefer progressive delivery such as blue-green or canary. I deploy in a controlled way, monitor health and error metrics, execute targeted smoke tests and shift traffic only after validation. ALB health checks help direct traffic to healthy targets. Rollback criteria are defined before deployment.

Remember: canary • blue-green • smoke test • ALB • health • traffic shift • rollback

6. CONTINUOUS MONITORING & AUTOMATED FEEDBACK

Objective: Continuously verify reliability and react to failures.

Say this: After deployment, monitoring continues through load-balancer health checks, logs, metrics, traces and APM. If agreed reliability thresholds are breached during the observation window, the delivery mechanism can trigger rollback to the previous known-good artifact. Findings then feed incident response and RCA.

Remember: observability • logs • metrics • traces • APM • thresholds • rollback • RCA

END-TO-END SPOKEN ANSWER

My CI/CD approach is a six-stage control loop. First, I shift security and quality left with local linting, secret detection and IaC checks. Second, the PR pipeline runs SCA, SAST, tests, coverage and Terraform plan, followed by peer approval. Third, after merge I build and scan one immutable artifact and publish it to the registry. Fourth, I deploy that same artifact to non-production and run DAST, functional, end-to-end and performance validation. Fifth, production uses controlled progressive delivery such as canary or blue-green, with smoke tests, health checks and predefined rollback criteria. Finally, continuous observability monitors logs, metrics, traces and business health. If agreed thresholds are breached, we mitigate or roll back, validate recovery and feed the findings into RCA and preventive improvements.

IMPORTANT L2 TRAPS

• Deployment succeeded does not mean the release succeeded; validate technical, functional and business health.
• Progressive delivery gates on successful validation and policy/approval, not simply “100% traffic”.
• Automatic rollback requires objective triggers, safe rollback mechanics and data/database compatibility.
• Promote the same tested immutable artifact; avoid rebuilding separately for production.
• In regulated/banking environments, automation does not remove mandatory approvals, segregation of duties, change records or audit evidence.

FAST MEMORY FLOW

PRE-COMMIT → PR/CI → IMMUTABLE ARTIFACT → NON-PROD VALIDATION → PROGRESSIVE PROD → OBSERVE → ROLLBACK/IMPROVE